

Technology Stack

Architecture, components and design decisions

Developing a Deep Learning-Based Computer Vision Framework for Automated
Pre- and Post-Rental Passenger Car Damage Inspection

MSc Dissertation Prototype — APIIT Sri Lanka

Stefania Crishani · CB016792

Generated 01 September 2026

1. Overview

The artefact is a two-tier web application: a Python service that owns the data, the inference and the reporting, and an Angular single-page dashboard that rental staff operate. The two communicate over a JSON REST API secured with JWT bearer tokens.

Layer	Technology	Version	Role
Presentation	Angular	17.3	Staff dashboard, standalone components
API	FastAPI	0.115	REST endpoints, validation, OpenAPI
ORM	SQLAlchemy	2.0	Declarative models, typed mappings
Database	SQLite	3.x	Development store; PostgreSQL-compatible
Inference	PyTorch / torchvision	2.5 / 0.20	EfficientNet-B3 classifier
Detection	Ultralytics YOLOv8	8.3	Phase 2 — written, not yet enabled
Imaging	Pillow	11.1	Validation, thumbnails, preprocessing
Reporting	ReportLab	4.2	Evidentiary PDF generation
Auth	python-jose, passlib/bcrypt	3.3 / 1.7	JWT issue-verify, password hashing
Migrations	Alembic	1.14	Versioned schema changes
Testing	pytest, httpx	8.3 / 0.28	20 API and integration tests

2. Architecture

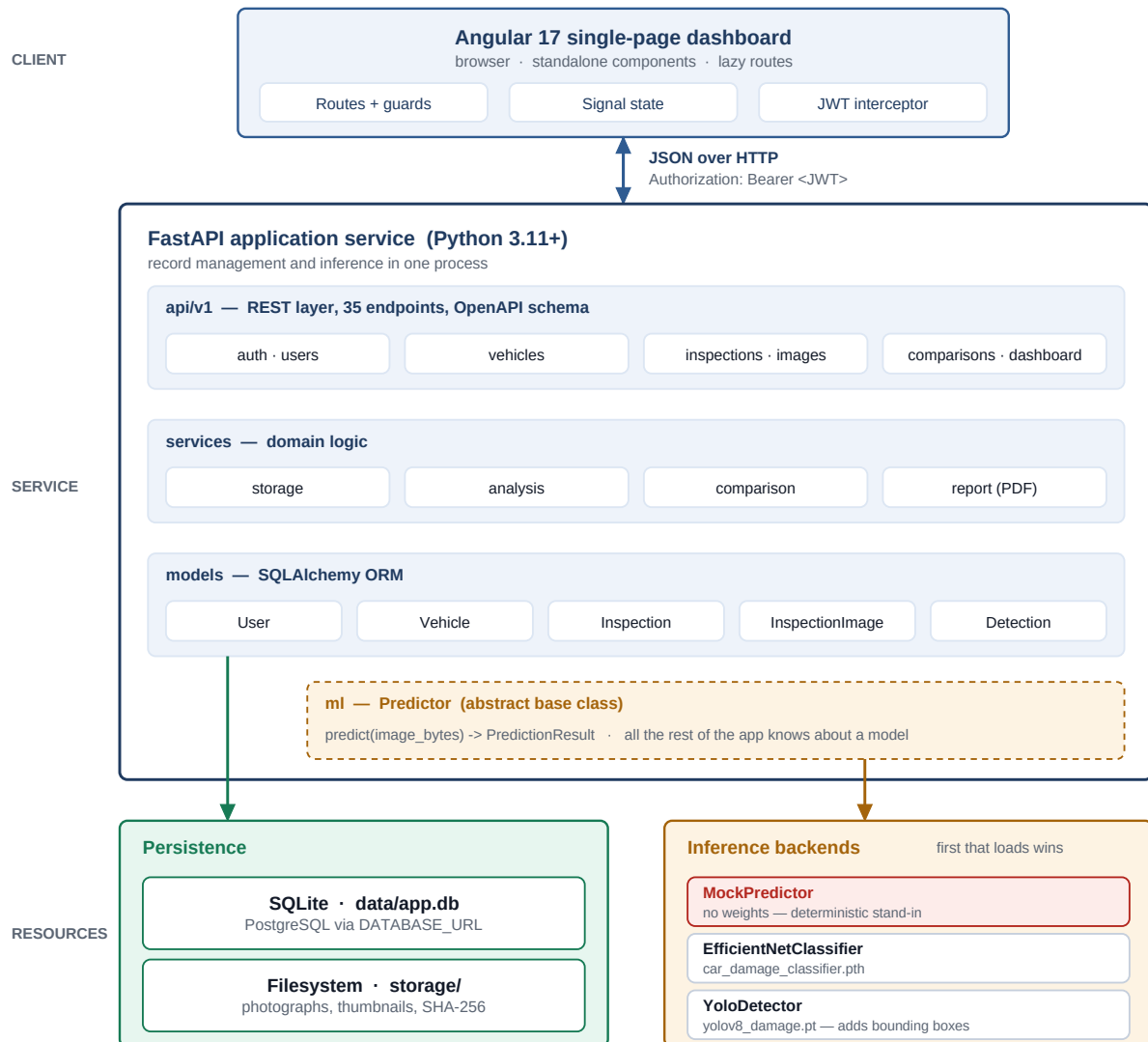


Figure 1. System architecture. The Angular client talks only to the FastAPI service; the service reaches persistence and inference through two narrow seams, either of which can be replaced without disturbing the other layers.

Every layer depends only on the layer beneath it. The API layer never touches the filesystem or a model directly; it calls a service. Services never import a concrete predictor; they call `get_predictor()` and receive something satisfying the `Predictor` interface.

3. Backend components

Package	Responsibility
app/api/v1/	Eight routers: auth, users, vehicles, inspections, images, comparisons, dashboard, system. 35 endpoints, all documented via OpenAPI.
app/core/	Settings (pydantic-settings), JWT and bcrypt helpers, and the damage taxonomy — class names, thresholds, colours, per-class test metrics.
app/db/	Engine and session factory, declarative base, schema creation and first-run seeding.
app/models/	Five entities: User, Vehicle, Inspection, InspectionImage, Detection.
app/schemas/	Pydantic request/response contracts, decoupled from the ORM so internal columns are never exposed by accident.
app/services/	storage (validate, hash, thumbnail), analysis (run inference, persist findings), comparison (pre/post delta), report (PDF generation).
app/ml/	The Predictor abstraction and its three implementations, plus the registry that selects one at startup.

Data model

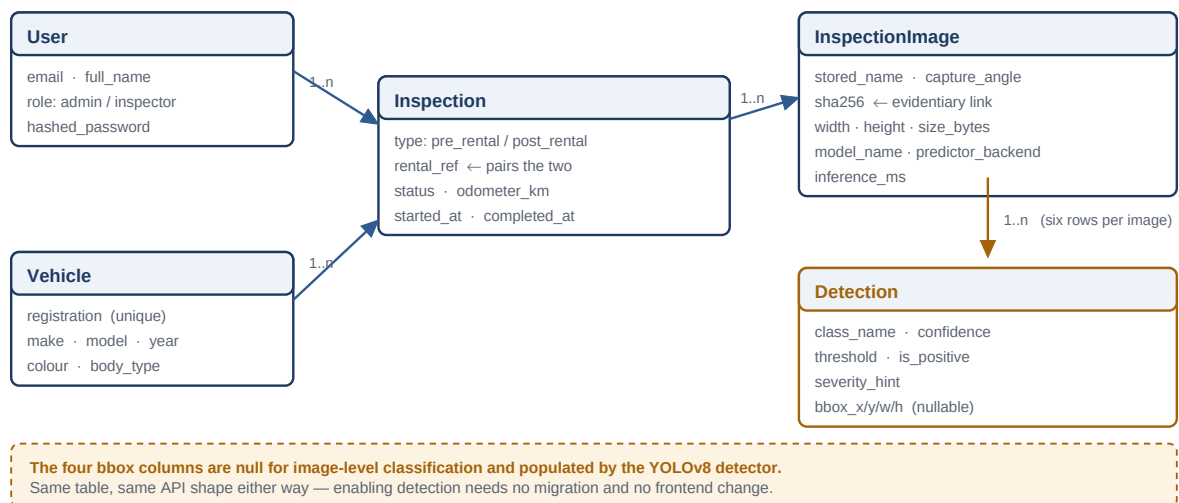


Figure 2. Data model. Five entities: every finding traces back through an image to an inspection, an inspector and a vehicle.

rental_ref is what pairs a pre-rental inspection with its post-rental counterpart, and is the key the comparison endpoint uses. sha256 is computed once at upload and never recomputed, giving each finding a verifiable link to the bytes analysed.

4. Frontend components

Area	Contents
core/	Typed API models mirroring the backend schemas; ApiService (all HTTP); AuthService (signal-based session state); a functional HTTP interceptor that attaches the bearer token and signs out on 401; route guards.
features/	Lazy-loaded routes: login, dashboard, fleet list and detail, inspection list / create / detail, pre-post comparison.
shared/	ConfidenceBar (score against its own threshold), DamageChip, StatusBadge, AuthImage and AnnotatedImage (fetch authenticated bytes, draw findings).
layout/	Application shell: sidebar navigation, top bar, responsive drawer.

Angular 17 features used deliberately:

- **Standalone components** throughout — no NgModules.
- **Signals** for component state, with `computed()` for derived values.
- **Functional interceptors and guards**, the modern replacement for class-based ones.
- **Lazy route loading** — each feature is a separate bundle; the initial download is 84 kB compressed.
- **Component input binding** from route parameters, avoiding manual `ActivatedRoute` plumbing.

5. The inference abstraction

This is the load-bearing design decision. Section 2.6 of the proposal invokes Sculley et al. (2015) on hidden technical debt: the risk is the model becoming *entangled* with the surrounding application, so that changing one forces changes to the other. The response here is a narrow interface — bytes in, a structured result out:

```
class Predictor(ABC):
    @abstractmethod
    def predict(self, image_bytes: bytes) -> PredictionResult: ...

    @property
    def is_real(self) -> bool: # False for stand-ins
        return True
```

Implementation	Weights required	Output	Status
MockPredictor	None	Deterministic scores from a hash of the bytes	Active by default
EfficientNetClassifier	car_damage_classifier.pth	Six image-level class scores	Ready; awaiting weights
YoloDetector	yolov8_damage.pt	Class scores with bounding boxes	Written; Phase 2

Selection is automatic: detector, then classifier, then mock — the first that loads wins. Consequently a clean checkout runs end-to-end with no weights, and gains real inference simply by a file appearing in `backend/models/`.

Why the mock is deterministic. Scores derive from a SHA-256 of the image bytes, so the same photograph always scores identically. Random scores would make the pre/post comparison meaningless and the test suite flaky. `is_real` is `False`, and every surface that renders a prediction — dashboard, inspection, comparison, both PDFs — reads that flag and warns the user.

6. Damage taxonomy and measured performance

Six CarDD classes (Wang et al., 2023). The figures below were **measured**, not transcribed: 643 images from the held-out test split were scored through the deployed predictor by `training/evaluate.py` on 2026-08-31T20:59:04Z.

Class	Thresh	Support	Precision	Recall	F1	AP	ROC-AUC
dent	0.687	157	0.809	0.834	0.821	0.896	0.962
scratch	0.641	183	0.750	0.885	0.812	0.888	0.954
crack	0.883	48	0.579	0.458	0.512	0.559	0.923
glass_shatter	0.955	71	1.000	0.915	0.956	0.980	0.987
lamp_broken	0.788	65	0.658	0.769	0.709	0.756	0.961
tire_flat	0.539	31	0.789	0.968	0.870	0.939	0.995
macro average			0.764	0.805	0.780	0.836	0.964

Exact-match ratio 0.743 (all six classes simultaneously correct) and any-damage accuracy 0.992 (damaged versus clean, the operationally relevant question). Inference latency 213 ms mean, 279 ms at the 95th percentile.

Training: 11 epochs run of 20 requested (early stopping), on mps, batch size 32, 9,296,100 trainable parameters. The retained checkpoint is epoch 6 (validation loss 0.2839); later epochs overfitted.

Per-class thresholds are not a refinement. Each class is reported positive only above its own F1-optimised cut-off, because a single flat 0.5 measurably underperforms on the rarer classes. The backend owns this table and serves it at `GET /api/v1/damage-classes`, so the dashboard renders thresholds, labels and colours from the same source the model is scored against and cannot drift from it.

7. Deviations from the proposal

Three architectural choices differ from Section 3.7 of the proposal. Each is deliberate and each is reversible.

7.1 The .NET 8 tier was removed

The proposal places a .NET 8 Web API between the Angular client and a FastAPI inference microservice. This implementation collapses those tiers into one Python service.

The isolation the proposal argues for is preserved: inference sits behind the `Predictor` interface, and nothing outside `app/ml/` knows which backend is answering. What is removed is a network hop and a language boundary that bought no isolation the interface does not already provide. What is genuinely lost is the .NET-to-Angular integration pattern named in research sub-question 3 — this should be stated in the dissertation as a scope decision rather than left for a reader to notice.

7.2 SQLite in place of SQL Server, local files in place of Azure Blob

Both are single-module swaps. The ORM layer avoids every SQLite-specific construct, so `DATABASE_URL` can point at PostgreSQL or SQL Server without touching a model. All file access goes through `app/services/storage.py`, so blob storage means reimplementing one module. The benefit is that the prototype needs no infrastructure at all to run — relevant for a supervisor demonstration or a usability study on a laptop.

7.3 Classification now, detection later

The proposal specifies YOLOv8 with localised bounding boxes; the notebook delivers image-level multi-label classification. The system is built so that gap closes at zero cost: `Detection` carries four nullable `bbox`

columns, the API returns `bbox: null` or a normalised box within the same response shape, and the annotated-image component already draws boxes when they are present. Enabling detection is a checkpoint file plus a restart — no migration, no API version change, no frontend change.

8. A defect found in the training notebook

While porting `car-damage-classification.ipynb` to a reproducible script, a substantive bug surfaced. The layer-unfreezing block intends to fine-tune the last three EfficientNet blocks:

```
if 'feature.6' in name or 'feature.7' in name or 'feature.8' in name:
    param.requires_grad = True
```

torchvision names these modules `features.6`, not `feature.6`. **The condition never matches.** The notebook's own output confirms it: 790,022 trainable parameters, which is exactly the replacement classifier head ($1536 \times 512 + 512 + 512 \times 6 + 6$) and nothing else.

Consequence. The entire convolutional backbone remained frozen, so no fine-tuning of visual features took place. The reported metrics are those of a *linear probe on frozen ImageNet features*, not of a fine-tuned model. This is the most likely explanation for crack performing worst (F1 0.47, AP 0.37): cracks depend most on adapting low-level texture representations, which is precisely what was never trained.

`backend/training/train_classifier.py` corrects this (matching on `features.{i}`) and is otherwise a faithful port. Retraining should improve the reported figures, and the before/after contrast is a legitimate result for the evaluation chapter.

8b. A dataset gap, and why it mattered more than the architecture

CarDD is a dataset for deciding *which* damage a photograph shows. Every one of its 4,000 images contains damage:

```
train: 2816 images, 2816 with damage annotations,    0 with none
val  :  810 images,  810 with damage annotations,    0 with none
test :  374 images,  374 with damage annotations,    0 with none
```

A model trained on it alone never observes an undamaged vehicle, so it has no representation of 'no damage'. Asked about a clean car it returns whichever class clears its threshold first — all six scores collapse toward 0.5, and the class with the lowest tuned threshold wins.

How it surfaced, and why the metrics hid it

The first trained model reported macro F1 0.786 and 'any-damage accuracy' 0.987 on the held-out split. Both were correctly computed. Both were blind to this failure, because **the test split contained no undamaged cars either**: a metric measured over exclusively-damaged images cannot express a false-alarm rate, so 'any-damage accuracy' was measuring recall alone while being labelled as though it measured both directions.

It surfaced when a photograph of an undamaged Ford Mustang was uploaded through the dashboard and returned *Dent*, 75.8%. Re-evaluating against 269 clean cars quantified it:

Metric	Damaged images only	With clean cars in the test set
Macro F1	0.786	0.677
dent precision	0.799	0.344
scratch precision	0.747	0.563
False alarms on clean cars	not measurable	238 / 269 (88.5%)

The correction

- **Negative examples added.** 2,681 photographs of undamaged vehicles (Stanford Cars) train with an all-zero label vector, split 70/20/10 in CarDD's own proportions so the false-alarm rate is measured on clean cars held out from training.
- **Loss weighting left alone.** Class weights are computed over the damaged subset only. Including the negatives would have roughly doubled every positive weight and pushed the model toward predicting damage more often — the opposite of the intent. The negatives contribute by example, not by reweighting.
- **The metric was renamed and gated.** `evaluate.py` reports an explicit false-alarm rate, and refuses to describe its headline figure as 'damaged versus clean' when no clean images are present. The dashboard shows the same, and warns when the rate is unmeasured rather than implying a pass.

For the dissertation. The proposal (Section 3.3) already anticipated this: it lists five classes including *no-damage/background*. CarDD supplies no such class. The gap between the annotation schema the proposal specified and the one the public dataset provides is a real, citable finding — and a concrete instance of the deployment-gap argument in Section 2.6, where a model with strong benchmark numbers fails on the first input a real operator would give it.

9. Security

Concern	Measure
Password storage	bcrypt via passlib; inputs capped at 72 bytes so the algorithm cannot silently truncate and hash a prefix
Session	HS256 JWT, 8-hour expiry, verified on every protected request
Authorisation	Two roles. Inspectors may read all records but modify only their own; user management and vehicle deletion are admin-only
Upload validation	Format determined by decoding the image, never from the client-supplied content type or filename extension
Path safety	Stored filenames are server-generated UUIDs; the client never supplies a path component
Media access	Photographs require a bearer token; the client fetches bytes over XHR and binds an object URL
CORS	Explicit origin allow-list, configurable per environment
Self-lockout	An administrator cannot deactivate their own account or change their own role

The default `SECRET_KEY` and both seeded passwords are development placeholders published in the source. They must be changed before the system runs anywhere reachable by others.

10. Quality and known limitations

Verification

- **20 backend tests**, all passing: authentication, role enforcement, registration normalisation, upload rejection, the full create-upload-analyse-report flow, analysis idempotency, mock determinism, and every comparison outcome.
- **Angular production build** clean, with no warnings or errors.
- **Alembic migration** verified to apply and roll back on a fresh database.

Limitations

Limitation	Impact and remedy
Synchronous inference	A large upload batch blocks the request; roughly 0.3–1 s per image on CPU with real weights. A task queue is the fix if the usability study exercises large batches.
Local filesystem storage	Not durable or shared across instances. Replace <code>app/services/storage.py</code> with a blob backend.
Severity is heuristic	Derived from threshold headroom; the model has no severity supervision. Labelled as a hint everywhere it appears, including in PDFs.
No cross-image deduplication	The same dent photographed from four angles counts as four pieces of evidence for that class.
Image-level findings only	Until YOLOv8 weights exist, a finding names a photograph, not a location on the panel.
Single-node deployment	SQLite and local files suit a prototype and a small usability study, not concurrent production use.